REST ASSURED NOTES

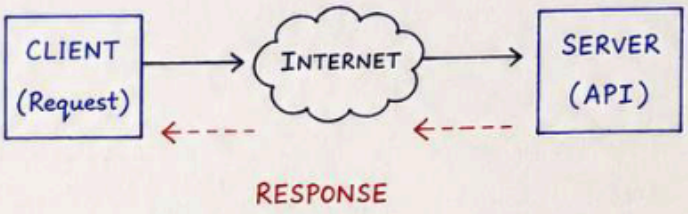
(API TESTING WITH REST ASSURED)

✳ What is API?

API (Application Programming Interface) is a set of rules that allows one application to talk to another application.

Example: Weather app gets data from server using API.

Client → Server → Response



REST vs SOAP

	REST	SOAP
1. Protocol	HTTP	HTTP / SMTP
2. Data Format	JSON, XML, etc.	XML only
3. Performance	Faster	Slower
4. Structure	Less Strict	More Strict
5. Use Case	Web, Mobile, Cloud	Enterprise Apps

HTTP Methods

- GET – Retrieve data from server
- POST – Create new resource
- PUT – Update entire resource
- PATCH – Update partial resource
- DELETE – Delete resource

REST Assured Introduction

- ✓ REST Assured is a Java library used for testing RESTful APIs.
- ✓ It provides a simple and fluent API to write tests.
- ✓ It is built on top of HTTP client.
- ✓ It supports different data formats like JSON, XML, Text, etc.

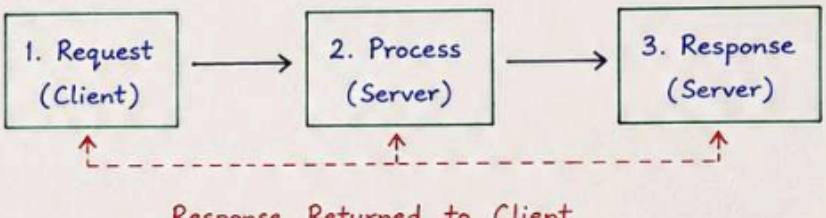
Why REST Assured?

- ✳ Easy to write and understand
- ✳ Fluent and readable API
- ✳ Less code
- ✳ Powerful validation support
- ✳ Good integration with TestNG / JUnit
- ✳ Supports JSON, XML Schema validation
- ✳ Actively maintained and widely used

Maven Dependency

```
<dependency>
    <groupId> io.rest-assured </groupId>
    <artifactId> rest-assured </artifactId>
    <version> 5.4.0 </version>
    <scope> test </scope>
</dependency>
```

Basic Request Flow



REST ASSURED NOTES

(PAGE 2) - HTTP REQUEST BASICS

Building Blocks of HTTP Request

1. Base URI → Protocol + Host + Port
2. Base Path → Common path for all requests
3. End Point → Specific resource / API
4. Headers → Additional information
5. Query Params → ? key = value
6. Path Params → Part of the URL
7. Request Body → Data to be sent
8. Content-Type → Type of request body
9. Accept → Expected response type

Common Headers

- Content-Type : application/json
- Accept : application/json
- Authorization : Bearer <token>
- User-Agent : RestAssured-Client
- Cache-Control : no-cache

Request Flow



Base URI, Base Path and End Point

- Base URI : https://reqres.in
- Base Path : /api
- End Point : /users/2

Complete URL : https://reqres.in/api/users/2

Query Params vs Path Params

- Query Params
  - Sent after ? in URL
  - Example : /users?page=2&limit=10
- Path Params
  - Part of the URL path
  - Example : /users/2

Simple GET Request Example

```
import static io.restassured.RestAssured.*;
public class GetRequestExample {
    public static void main(String[] args) {
        given()
            .baseUri("https://reqres.in")          ← Base URI (protocol + host + port)
            .basePath("/api")                      ← Base Path (common path for all requests)
            .queryParam("page", "2")               ← Query Parameter added in URL
            .header("Accept", "application/json")  ← Header added in request
        when()
            .get("/users")                         ← Sending GET request to /users
        then()
            .statusCode(200)                       ← Validate status code
            .log().prettyPrint();                  ← Print response in readable format
    }
}
```

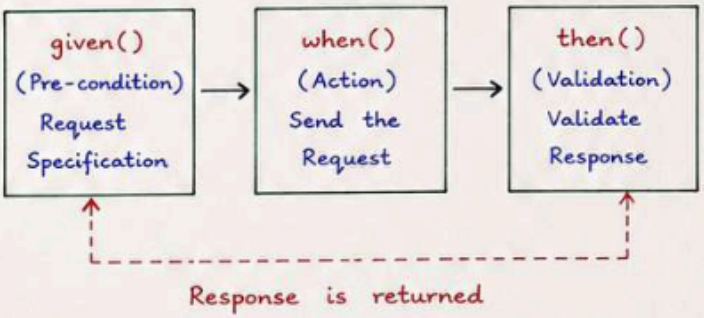
REST ASSURED NOTES

HTTP METHODS & SENDING REQUESTS

✳ HTTP Methods in REST Assured

- GET – Retrieve data from server.
- POST – Create a new resource.
- PUT – Update entire resource.
- PATCH – Update partial resource.
- DELETE – Delete a resource.

Request Builder Flow



1. GET Request

```
given()
      .baseUri("https://reqres.in")
      .basePath("/api")
      .queryParam("page", "2")
      .header("Accept", "application/json")
when()
      .get("/users")
then()
      .statusCode(200)
      .log().prettyPrint();
```

- Used to fetch data.
- Should not modify data on server.
- Data is sent in URL (Query Params).

Example URL :
https://reqres.in/api/users?page=2

2. POST Request

```
given()
    .baseUri("https://reqres.in")
    .basePath("/api")
    .header("Content-Type", "application/json")
    .body("{\"name\":\"morpheus\",\"job\":\"leader\"}")
when()
    .post("/users")
then()
    .statusCode(200)
    .log().prettyPrint();
```

- Used to create a new resource.
- Data is sent in request body.

3. PUT Request

```
given()
      .baseUri("https://reqres.in")
      .basePath("/api")
      .header("Content-Type", "application/json")
      .body("{\"name\":\"morpheus\",\"job\":\"zion resident\"}")
when()
      .put("/users/2")
then()
      .statusCode(200)
      .log().prettyPrint();
```

- Used to update entire resource.
- Sends data in request body.

4. PATCH Request

```
given()
    .baseUri("https://reqres.in")
    .basePath("/api")
    .header("Content-Type", "application/json")
    .body("{\"job\":\"zion resident\"}")
when()
    .patch("/users/2")
then()
    .statusCode(200)
    .log().prettyPrint();
```

- Used to update partial resource.
- Only fields included are updated.

5. DELETE Request

```
given()
      .baseUri("https://reqres.in")
      .basePath("/api")
      .header("Accept", "application/json")
when()
      .delete("/users/2")
then()
      .statusCode(204)
      .log().prettyPrint();
```

- Used to delete a resource.
- Usually returns 204 (No Content).

Key Points

✳ when() performs the action (HTTP method).

✳ then() is used to validate the response.

✳ Always validate status code.

✳ Use proper headers and content type.

✳ Use log().all() or log().prettyPrint() to see request & response.

REST ASSURED NOTES

RESPONSE VALIDATION BASICS

1. Status Code Validation

- Used to validate the response status code.

```
then().statusCode(200);        // exact match
then().statusCode(not(404));
then().statusCode(lessThan(400));
then().statusCode(greaterThan(199));
```

Common Status Codes

- 200 – OK
- 201 – Created
- 204 – No Content
- 400 – Bad Request
- 401 – Unuathorized
- 403 – Forbidden
- 404 – Not Found
- 500 – Internal Server Error

---

2. Status Line Validation

- Validates the entire status line.

```
then().statusLine("HTTP/1.1 200 OK");
```

3. Headers Validation

- Used to validate response headers.

```
then().header("Content-Type", "application/json");
then().header("Server", containsString("Apache"));
then().header("Content-Length", notNullValue());
then().headers("Cache-Control", "no-cache",
               "Pragma", "no-cache");
```

Common Headers

- Content-Type
- Content-Length
- Server
- Cache-Control
- Set-Cookie
- Authorization

4. Body Validation

- Validate the response body content.

```
then().body(containsString("success"));
then().body("name", equalTo("John"));
then().body("id", greaterThan(0));
then().body("email", matchesPattern(".+@.+\\..+"));
```

Note:
Body validation is usually done using JSON Path and Matchers.

5. Response Time Validation

- Used to validate the time taken for response.

```
then().time(lessThan(2000L));    // in milliseconds
then().time(lessThan(2L), TimeUnit.SECONDS);
```

6. Extract Response

- Used to extract data from response.

```
Response res = given().get("/users/1");
String name = res.jsonPath().getString("name");
int id = res.jsonPath().getInt("id");
List<String> emails = res.jsonPath().getList("emails");
```

Response Object Methods

- response().asString() – Get response as String
- response().asPrettyString() – Get response in pretty format
- response().statusCode() – Get status code
- response().statusLine() – Get status line
- response().header("key") – Get specific header value
- response().headers() – Get all headers
- response().time() – Get time taken
- response().cookie("name") – Get cookie value

Pretty Print Example

```
given()
    .get("/users/1")
  .then()
    .log().all();        // prints request + response
    .log().body();       // prints only response body
    .log().headers();    // prints only headers
```

REST ASSURED NOTES

ASSERTIONS & JSON PATH

✳ Assertions in REST Assured

- Assertions are used to validate the response.
- REST Assured uses Hamcrest Matchers for validations.
- We use then().body() for body validation and then().header() for header validation.

Example JSON Response

```
{
   "id": 1,
   "name": "John Doe",
   "email": "john@example.com",
   "age": 30,
   "isActive": true,
   "roles": ["admin", "user"],
   "address": {
      "city": "New York",
      "zip": "10001"
   }
}
```

JSON Path in REST Assured

- JSON Path is used to navigate through JSON response and extract data.
- It helps to access nested elements easily.

JSON Path Syntax

```
$              - Root element
$.key          - Access key
$.key1.key2    - Access nested key
$[index]       - Access array item
$.key[index]   - Access array inside key
```

Common Hamcrest Matchers

- ✳ equalTo() - Matches the exact value
- ✳ notEqualTo() - Not matches the value
- ✳ containsString() - Checks substring exists
- ✳ startsWith() - Checks starting text
- ✳ endsWith() - Checks ending text
- ✳ hasItem() - Checks item in list
- ✳ notNullValue() - Checks value is not null
- ✳ nullValue() - Checks value is null
- ✳ greaterThan() - Checks value is greater
- ✳ lessThan() - Checks value is less
- ✳ hasSize() - Validates size of collection

Assertion Examples

```
// Validate ID
then().body("id", equalTo(1));
// Validate Name
then().body("name", equalTo("John Doe"));
// Validate Email contains
then().body("email", containsString("example.com"));
// Validate Age greater than
then().body("age", greaterThan(18));
// Validate isActive is true
then().body("isActive", equalTo(true));
// Validate roles has item
then().body("roles", hasItem("admin"));
// Validate city in address
then().body("address.city", equalTo("New York"));
// Validate zip code
then().body("address.zip", equalTo("10001"));
// Validate roles size
then().body("roles", hasSize(2));
```

JSON Path Examples

```
String name = response.jsonPath().getString("name");          // "John Doe"
int id = response.jsonPath().getInt("id");                    // 1
String email = response.jsonPath().getString("email");        // "john@example.com"
boolean active = response.jsonPath().getBoolean("isActive");  // true
String city = response.jsonPath().getString("address.city");  // "New York"
String zip = response.jsonPath().getString("address.zip");    // "10001"
String role0 = response.jsonPath().getString("roles[0]");     // "admin"
List<String> roles = response.jsonPath().getList("roles");    // [admin, user]
```

Important Points

- ✳ JSON Path is case sensitive.
- ✳ Use correct path to avoid null values.
- ✳ getString(), getInt(), getBoolean(), getList() are commonly used.
- ✳ JSON Path works on response as well as request body.

REST ASSURED NOTES

AUTHENTICATION & REQUEST SPECIFICATION

✳ Authentication in REST Assured

APIs may require authentication to access secured resources.

1. Basic Authentication

```
given()
    .auth().basic("admin", "password")
    .baseUri("https://reqres.in")
when()
    .get("/api/users")
then()
    .statusCode(200);
```

2. Bearer Token Authentication

```
given()
    .auth().oauth2("YOUR_ACCESS_TOKEN")
    .baseUri("https://api.example.com")
when()
    .get("/users")
then()
    .statusCode(200);
```

3. OAuth2 (Client Credentials)

Client ⟶ Auth Server ⟶ Access Token ⟶ API
(Request token) (Use token)

Steps :
1. Get access token from auth server.
2. Use token in request (Bearer Token).

✳ Request Specification

- Used to define common request details.
- Helps to avoid repetition.
- Improves readability and maintainability.

Example :

```
RequestSpecification reqSpec = given()
    .baseUri("https://reqres.in")
    .basePath("/api")
    .contentType("application/json")
    .accept("application/json")
    .header("Accept-Language", "en-US")
    .log().all()
    .build();
```

Using RequestSpecification

```
given()
    .spec(reqSpec)
    .pathParam("id", 2)
when()
    .get("/users/{id}")
then()
    .statusCode(200);
```

✳ Response Specification

- Used to define expected response details.
- Helps in reusability of validations.

```
ResponseSpecification resSpec = expect()
    .statusCode(200)
    .contentType("application/json")
    .time(lessThan(2000L))
    .log().body()
    .build();
```

Using ResponseSpecification

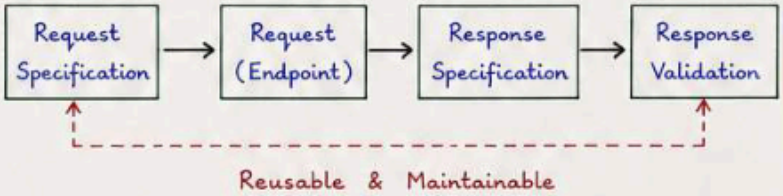
```
given()
    .spec(reqSpec)
when()
    .get("/users/2")
then()
    .spec(resSpec);
```

Summary :
Use Specifications to make your tests clean, reusable, and easy to maintain.

✳ Difference : Without vs With Specifications

Without Specification	With Specification
Code repetition	Reusable and clean code
Hard to maintain	Easy to maintain
More prone to errors	Less prone to errors
Hard to update common details	Update once, all tests will use it
Less readable	More readable

Overall Flow with Specifications

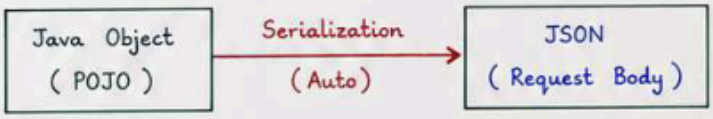


REST ASSURED NOTES

SERIALIZATION & DESERIALIZATION

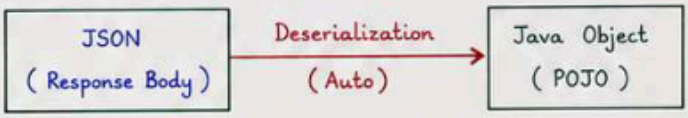
✳ What is Serialization?

- Process of converting Java object into JSON format (String) to send in request body.
- Done automatically by REST Assured using Jackson.
- We just pass Java object to .body() and it converts it to JSON.



✳ What is Deserialization?

- Process of converting JSON response into Java object.
- Done automatically by REST Assured using Jackson.
- We use .as(Class.class) to convert JSON into Java object.



✳ POJO Class

- Plain Old Java Object (POJO).
- Normal Java class with private variables.
- No special annotation required (or use Jackson annotations if field names are different).
- Must have default constructor, getters and setters.

```
public class User {
    private int id;
    private String name;
    private String email;

    public User() {}

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }

    // getter and setter for name, email
}
```

✳ Serialization Example (Sending POJO in Request)

```
User user = new User();
user.setId(1);
user.setName("John Doe");
user.setEmail("john@example.com");

given()
    .contentType("application/json")
    .body(user)      // Java object will be converted to JSON
when()
    .post("/users")
then()
    .statusCode(201);
```

✳ Deserialization Example (Convert JSON to POJO)

```
User responseUser = given()
    .when()
    .get("/users/1")
    .then()
    .statusCode(200)
    .extract()
    .as(User.class);    // JSON converted to Java Object

System.out.println(responseUser.getName());
System.out.println(responseUser.getEmail());
```

✳ Using Jackson Annotations (If JSON field names are different)

```
import com.fasterxwk.jackson.annotation.JsonProperty;

public class User {
    @JsonProperty("user_id")
    private int id;

    @JsonProperty("user_name")
    private String name;

    // getters and setters
}
```

✳ Key Points

- ✳ REST Assured uses Jackson library for (de)serialization.
- ✳ No need to write manual conversion code.
- ✳ Use POJO classes with getters and setters.
- ✳ Use @JsonProperty if JSON field names are different.
- ✳ Serialization → Java Object to JSON (for request).
- ✳ Deserialization → JSON to Java Object (for response).
- ✳ Makes code clean, readable and type safe.

REST ASSURED NOTES

COMMON METHODS & BEST PRACTICES

✳ Common REST Assured Methods

Method	Description	Example
given()	Start building the request	given()
baseUri()	Set base URI	baseUri("https://api.com")
basePath()	Set base path	basePath("/v1")
header()	Add header	header("Content-Type", "application/json")
param()	Add query parameter	param("page", 1)
queryParam()	Add query parameter (preferred)	queryParam("page", 1)
pathParam()	Add path parameter	pathParam("id", 10)
body()	Add request body	body(payload)
when()	Trigger the request	when()
get()	Send GET request	get("/users")
post()	Send POST request	post("/users")
put()	Send PUT request	put("/users/1")
patch()	Send PATCH request	patch("/users/1")
delete()	Send DELETE request	delete("/users/1")
then()	Start verifying response	then()
statusCode()	Validate status code	statusCode(200)
body()	Validate response body	body("name", equalTo("John"))
header()	Validate response headers	header("Server", containsString("Apache"))
extract()	Extract data from response	extract().path("id")
response()	Get response object	response()
prettyPrint()	Print response in readable format	prettyPrint()
log().all()	Log request & response	log().all()
cookie()	Add or validate cookies	cookie("token", "abc123")

✳ Best Practices

- ✓ Always validate status code first.
  Ex: statusCode(200)
- ✓ Validate headers and body for better test coverage.
- ✓ Use meaningful endpoint names and reusable code.
- ✓ Use RequestSpecification to avoid code duplication.
- ✓ Extract data and reuse in further requests.
- ✓ Validate response time for performance.
  Ex: time(lessThan(2000L))
- ✓ Use JSON Path for dynamic data validation.
- ✓ Use schema validation to ensure response structure is correct.
- ✓ Keep test data and payloads separate (files / objects).
- ✓ Maintain independence between tests.
- ✓ Handle negative scenarios and error responses.
- ✓ Always close resources and avoid hardcoding values.

✳ Common Logging Levels

- → log().all() : Logs request and response details
- → log().headers() : Logs headers
- → log().body() : Logs body
- → log().params() : Logs query/path parameters
- → log().cookies() : Logs cookies
- → prettyPrint() : Prints response in readable JSON format

✳ Example Flow (Simple GET Request)

```
given()
    .baseUri("https://reqres.in")          } Build
    .basePath("/api")                      } Request
    .queryParam("page", 2)
when()                                     } Send
    .get("/users")                         } Request
then()
    .statusCode(200)                       } Validate
    .header("Content-Type", "application/json")  } Response
    .body("data[0].first_name", equalTo("Michael"))
    .log().all();
```

✳ Useful Extract Methods

Method	Description	Example
extract().path()	Extract single value	extract().path("id")
extract().jsonPath()	Get JsonPath object	extract().jsonPath()
extract().response()	Get full response	extract().response()
extract().asString()	Get response as String	extract().asString()
extract().as(Class)	Convert JSON to POJO	extract().as(User.class)
getStatusCode()	Get status code	getStatusCode()
getTime()	Get response time (ms)	getTime()

✳ API Lifecycle in REST Assured

